

Desarrollo: Cliente-Servidor, Frontend/Backend y APIs

1. Del Cliente al Servidor

Paso 1. El usuario escribe `www.youtube.com` en el navegador y presiona Enter. El navegador (cliente) revisa su caché y, si no conoce la dirección IP, consulta al DNS.

Paso 2. El DNS (Domain Name System) traduce el nombre de dominio (`www.youtube.com`) a una dirección IP, que identifica al servidor en Internet.

Paso 3. Con la dirección IP, el navegador establece la conexión con el servidor.

Paso 4. Se utiliza HTTPS para crear una conexión segura mediante TLS/SSL, cifrando la información intercambiada.

Paso 5. El servidor recibe la solicitud HTTP/HTTPS, procesa la petición, consulta sus recursos y prepara la respuesta.

Paso 6. El servidor responde con HTML, CSS, JavaScript y demás recursos necesarios.

Paso 7. El navegador interpreta esos archivos y construye la página.

Paso 8. Cuando el usuario reproduce un video, el navegador solicita los fragmentos del video mediante streaming, permitiendo su reproducción inmediata.

Diagrama del proceso

Usuario



Navegador (Cliente)



Consulta DNS



DNS —► Devuelve IP



Conexión HTTPS



Servidor



Procesa solicitud



Respuesta HTTP



HTML/CSS/JS



Streaming de video



Video en pantalla

2. Frontend y Backend en acción

Frontend: interfaz con la que interactúa el usuario (formularios, calendario de citas, botones, historial).

Tecnologías Frontend: HTML, CSS, React (también JavaScript, Angular o Vue.js).

Backend: procesa la lógica de negocio, valida usuarios, guarda citas y se comunica con la base de datos.

Tecnologías Backend: Python (Django/Flask), Java (Spring Boot), Node.js (Express).

El frontend se comunica con el backend mediante una API utilizando HTTP. Envía un request (por ejemplo, POST para crear una cita) y recibe un response con el resultado, normalmente en formato JSON.

3. REST vs SOAP vs GraphQL

Tipo de API	Formato de datos	Flexibilidad	Dificultad	Uso actual
REST	JSON/XML	Media	Baja	Alta
SOAP	XML	Baja	Alta	Media
GraphQL	JSON	Alta	Media	Alta

¿Cuál es más apropiada para una startup moderna?

REST es la opción más recomendable porque es sencilla de implementar, ampliamente utilizada, tiene abundante documentación y facilita el desarrollo rápido. Utiliza HTTP y normalmente JSON, lo que permite una integración sencilla con aplicaciones web y móviles. GraphQL es muy flexible, pero suele ser más conveniente en sistemas grandes y complejos, mientras que SOAP es más adecuado para entornos empresariales con requisitos estrictos de seguridad y estandarización.

Explorando APIs con Postman

Parte 1: Selección de la API

Nombre de la API: JSONPlaceholder

Descripción: JSONPlaceholder es una API REST gratuita utilizada para pruebas y aprendizaje. Permite simular operaciones CRUD (consultar, crear, actualizar y eliminar recursos) sobre publicaciones, comentarios, usuarios, álbumes y tareas sin necesidad de autenticación.

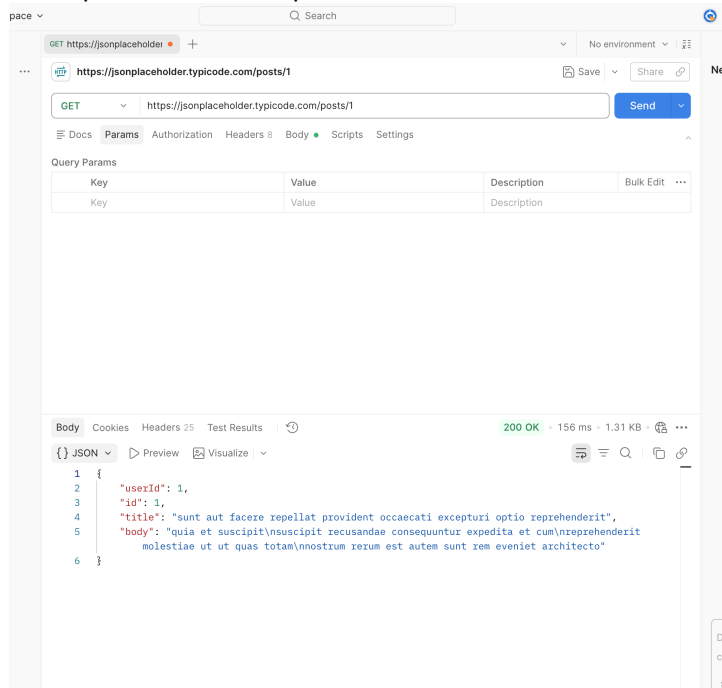
Parte 2: Configuración en Postman

Colección: JSONPlaceholder API

Solicitud 1 - GET

Endpoint: <https://jsonplaceholder.typicode.com/posts/1>

Descripción: Obtiene la publicación con ID 1.



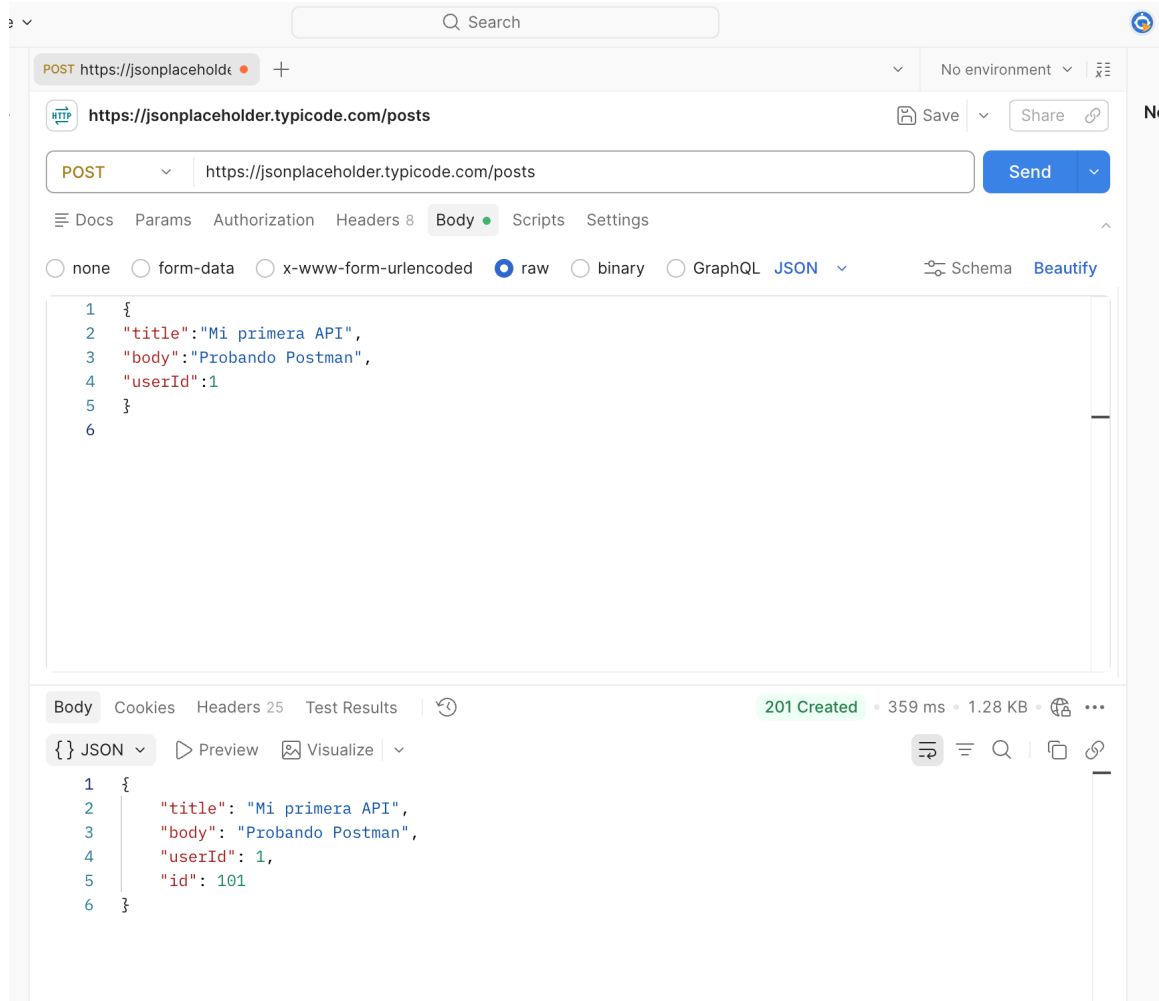
Solicitud 2 - POST

Endpoint: <https://jsonplaceholder.typicode.com/posts>

Body:

```
{
  "title": "Mi primera API",
  "body": "Probando Postman",
  "userId": 1
}
```

Descripción: Simula la creación de una nueva publicación.



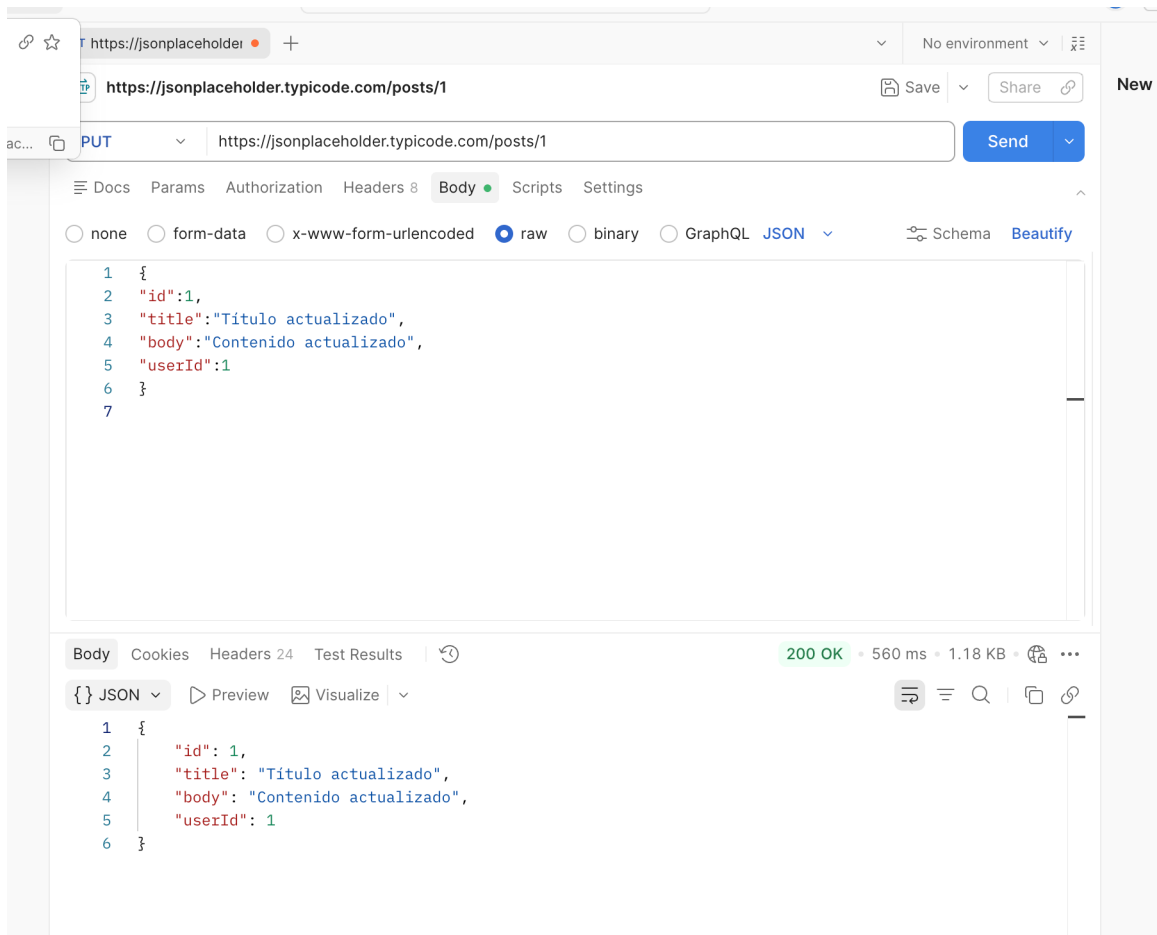
Solicitud 3 - PUT

Endpoint: <https://jsonplaceholder.typicode.com/posts/1>

Body:

```
{
  "id": 1,
  "title": "Título actualizado",
  "body": "Contenido actualizado",
  "userId": 1
}
```

Descripción: Actualiza completamente una publicación.



Environment

No requiere autenticación. Opcionalmente puede crearse la variable `base_url` con el valor <https://jsonplaceholder.typicode.com>

Parte 3: Ejecución y análisis

Solicitud	Estado	Respuesta	Header relevante
GET	200 OK	<code>{"userId":1,"id":1,...}</code>	Content-Type: application/json
POST	201 Created	<code>{"id":101,...}</code>	Content-Type: application/json
PUT	200 OK	<code>{"id":1,"title":"Título actualizado",...}</code>	Content-Type: application/json

Parte 4: README

Descripción general:

JSONPlaceholder es una API REST gratuita para practicar el consumo de servicios web.

GET /posts/1

- Método: GET
- Endpoint: /posts/1
- Respuesta: Devuelve una publicación en formato JSON.

POST /posts

- Método: POST
- Endpoint: /posts
- Body: título, body y userId.
- Respuesta: Devuelve el recurso creado con un nuevo id.

PUT /posts/1

- Método: PUT
- Endpoint: /posts/1
- Body: datos actualizados.
- Respuesta: Devuelve el recurso actualizado.

Aprendizaje:

Aprendí cómo utilizar los métodos HTTP, interpretar respuestas JSON y comprender el intercambio de información entre cliente y servidor mediante APIs REST.

Parte 5: Reflexión final

Aprendí que las APIs permiten que distintas aplicaciones se comuniquen mediante solicitudes HTTP. Cada método (GET, POST, PUT y DELETE) cumple una función específica y el servidor responde con códigos de estado y datos generalmente en formato JSON.

Postman me ayudó a visualizar todo el proceso de comunicación entre cliente y servidor. Pude enviar solicitudes, revisar los códigos de respuesta, analizar los headers y entender cómo funcionan las APIs REST en aplicaciones modernas.